Lab Report: Lab 2 - Layered Architecture Design (Logical View)

Kyte

Contents

1. Cover Page
2. Abstract/Summary
3. Lab Specific Section: II. Layered Architecture Design 3.1 Activity Practice 1: Defining Layers and Responsibilities 3.1.1 The Four Layers Definition 3.1.2 Data Flow Definition 3.2 Activity Practice 2: Component Identification (Kanban Board Feature) 3.2.1 Component Identification for Core Features 3.2.2 Interface Definitions 3.3 Activity Practice 3: Component Diagram Modeling 3.3.1 Component Diagram Description 3.3.2 Diagram Key Elements 3.3.3 ASR Mapping to Components
4. Conclusion & Reflection

1. Cover Page

Title: Layered Architecture Design for the Kyte Project Management Tool

Course Info: Software Architecture

Student Details:

Họ và Tên	MSSV
Trần Thành Long	23010070
Nguyễn Xuân Mạnh	23010045

Date: Dec 11, 2025

2. Abstract/Summary

This lab focuses on designing the Layered Architecture for the Kyte application, representing the system's static structure and component relationships. Building upon the

requirements elicited in Lab 1, this design phase formally defines the four architectural layers and their responsibilities.

The architecture is driven by the Architecturally Significant Requirements (ASRs) identified previously, particularly Modifiability (ASR-1), Security (ASR-2), Reliability (ASR-3), Scalability (ASR-4), and Performance (ASR-5). The Layered Architecture pattern enforces strict separation of concerns and downward-only dependencies, enabling future extensibility (e.g., Mobile App development) while maintaining security and data integrity.

Key deliverables include the formal layer definitions, component identification for the core 'Move Task' feature, and a UML Component Diagram modeling the logical view of the system.

3. Lab Specific Section: II. Layered Architecture Design

3.1 Activity Practice 1: Defining Layers and Responsibilities

Goal: Formally define the four layers and their roles within the Kyte monolithic structure.

3.1.1 The Four Layers Definition

Adhering to the Layered Pattern Rule (a layer can only interact with the layer directly below it):

Layer	Purpose/Responsibility	Output/Artifact
1. Presentation Layer (UI/API)	Handles HTTP requests, session cookie authentication (Better Auth), and serves the RESTful API endpoints. Routes incoming requests to appropriate controllers. Enforces stateless design (ASR-4).	Controllers: BoardController, TaskController, ColumnController
2. Business Logic Layer (Service/Domain)	Contains the core business rules, validation logic, permission checks (ASR-2), optimistic locking conflict resolution (ASR-3), and transaction management. Orchestrates data access and	Services: BoardService, TaskService, ColumnService, AuthService

Layer	Purpose/Responsibility	Output/Artifact
	enforces Kanban workflow rules.	
3. Persistence Layer (Data Access)	Responsible for mapping business objects to database entities and executing CRUD operations via Drizzle ORM. Implements versioning/timestamps for conflict detection (ASR-3). Manages database indexing strategies (ASR-5).	Repositories: BoardRepository, TaskRepository, ColumnRepository, UserRepository
4. Data Layer	The physical database storage system (PostgreSQL 18). Contains the actual tables, indexes, and constraints that persist the Kyte data.	Database Schema (Tables: workspaces, boards, columns, tasks, users, board_members, comments)

3.1.2 Data Flow Definition

Scenario: A Board Member moves a Task from 'In Progress' to 'Done' column.

Request Flow:

3.2 Activity Practice 2: Component Identification (Kanban Board Feature)

Goal: Break down the core Kanban features into concrete components that reside in the top three layers.

3.2.1 Component Identification for Core Features

Feature 1: Move Task (Critical - FR-03) - Layer 1 (Presentation): - Component Name: TaskController - Responsibility: Receives request PATCH /tasks/{id}/move. Validates authentication session. Calls Business Logic Layer. Returns JSON response. - Layer 2 (Business Logic): - Component Name: TaskService - Responsibility: Receives task ID, target column ID, and position. Validates user has board membership. Checks version for optimistic locking (ASR-3). Updates task's column assignment. - Layer 3 (Persistence): - Component Name: TaskRepository - Responsibility: Translates request into database query with version check: UPDATE tasks SET column_id = ?, position = ?, version = version + 1 WHERE id = ? AND version = ?. Returns updated entity.

Feature 2: Manage Board (FR-01) - Layer 1 (Presentation): - Component Name: BoardController - Responsibility: Receives requests like POST /boards, GET /boards/{id}, PUT /boards/{id}. Handles request validation and routing. - Layer 2 (Business Logic): - Component Name: BoardService - Responsibility: Enforces board creation rules. Validates user permissions for board access (ASR-2). Manages board visibility settings (FR-06). - Layer 3 (Persistence): - Component Name: BoardRepository - Responsibility: Executes CRUD operations: INSERT INTO boards, SELECT * FROM boards WHERE id = ?, etc.

Feature 3: Manage Columns (FR-01) - Layer 1 (Presentation): - Component Name: ColumnController - Responsibility: Receives requests like POST /boards/{boardId}/columns, PUT /columns/{id}, DELETE /columns/{id}. - Layer 2 (Business Logic): - Component Name: ColumnService - Responsibility: Validates column operations within board context. Enforces ordering logic. Checks user board membership. - Layer 3 (Persistence): - Component Name: ColumnRepository - Responsibility: Translates to queries: INSERT INTO columns, UPDATE columns SET position = ?, DELETE FROM columns WHERE id = ?.

Feature 4: Manage Tasks (FR-02) - Layer 1 (Presentation): - Component Name: TaskController - Responsibility: Receives requests for task CRUD: POST /tasks, GET /tasks/{id}, PATCH /tasks/{id}. - Layer 2 (Business Logic): - Component Name: TaskService - Responsibility: Enforces task validation rules (title required, due date format). Manages task details (labels, attachments, checklists, assignees). - Layer 3 (Persistence): - Component Name: TaskRepository - Responsibility: Executes task CRUD with proper indexing on column_id and board_id for performance (ASR-5).

Feature 5: Authentication & Authorization (ASR-2) - Layer 1 (Presentation): - Component Name: AuthPlugin - Responsibility: Intercepts all requests. Validates session cookies. Extracts user context and attaches to request. - Layer 2 (Business Logic): - Component Name: AuthService - Responsibility: Handles login/logout logic. Manages sessions (stateless for ASR-4). Validates user credentials. - Component Name: PermissionService - Responsibility: Validates board membership. Checks if user has required role (Member vs Admin). - Layer 3 (Persistence): - Component Name: UserRepository - Responsibility: Queries user data: SELECT * FROM users WHERE email = ?. - Component Name: BoardMemberRepository - Responsibility: Queries membership: SELECT * FROM board_members WHERE board_id = ? AND user_id = ?.

3.2.2 Interface Definitions

3.2.2.1 Interfaces provided by Business Logic Layer to Presentation Layer:

ITaskService Interface

```
interface ITaskService {
  getTaskDetails(taskId: string, userId: string): Promise<Task>
  createTask(columnId: string, taskData: CreateTaskDTO, userId: string):
        Promise<Task>
  updateTask(taskId: string, taskData: UpdateTaskDTO, userId: string):
        Promise<Task>
  moveTask(taskId: string, targetColumnId: string, position: string, userId:
        string, version: number): Promise<Task>
  deleteTask(taskId: string, userId: string): Promise<void>
}
```

IBoardService Interface

```
interface IBoardService {
  createBoard(boardData: CreateBoardDTO, userId: string): Promise<Board>
  getBoardDetails(boardId: string, userId: string): Promise<Board>
  updateBoard(boardId: string, boardData: UpdateBoardDTO, userId: string):
        Promise<Board>
  inviteMember(boardId: string, inviteeEmail: string, userId: string):
        Promise<void>
  updateVisibility(boardId: string, visibility: 'private' | 'public', userId:
        string): Promise<Board>
}
```

IColumnService Interface

```
interface IColumnService {
  createColumn(boardId: string, columnData: CreateColumnDTO, userId: string):
        Promise<Column>
  updateColumn(columnId: string, columnData: UpdateColumnDTO, userId:
        string): Promise<Column>
  reorderColumns(boardId: string, listOrder: string[], userId: string):
        Promise<Column[]>
  deleteColumn(columnId: string, userId: string): Promise<void>
}
```

IAuthService Interface

```
interface IAuthService {
  login(email: string, password: string): Promise<AuthToken>
  validateToken(session: string): Promise<User>
  logout(session: string): Promise<void>
}
```

3.2.2.2 Interfaces provided by Persistence Layer to Business Logic Layer:

ITaskRepository Interface

```
interface ITaskRepository {
  findById(taskId: string): Promise<TaskEntity | null>
  findByColumnId(columnId: string): Promise<TaskEntity[]>
  create(cardEntity: TaskEntity): Promise<TaskEntity>
  update(cardEntity: TaskEntity): Promise<TaskEntity>
  updateWithVersionCheck(taskId: string, updates: Partial<TaskEntity>,
         expectedVersion: number): Promise<TaskEntity | null>
  delete(taskId: string): Promise<void>
}
```

IBoardRepository Interface

```
interface IBoardRepository {
  findById(boardId: string): Promise<BoardEntity | null>
  findByUserId(userId: string): Promise<BoardEntity[]>
  create(boardEntity: BoardEntity): Promise<BoardEntity>
  update(boardEntity: BoardEntity): Promise<BoardEntity>
  delete(boardId: string): Promise<void>
}
```

IColumnRepository Interface

```
interface IColumnRepository {
  findById(columnId: string): Promise<ColumnEntity | null>
  findByBoardId(boardId: string): Promise<ColumnEntity[]>
  create(listEntity: ColumnEntity): Promise<ColumnEntity>
  update(listEntity: ColumnEntity): Promise<ColumnEntity>
  delete(columnId: string): Promise<void>
}
```

IBoardMemberRepository Interface

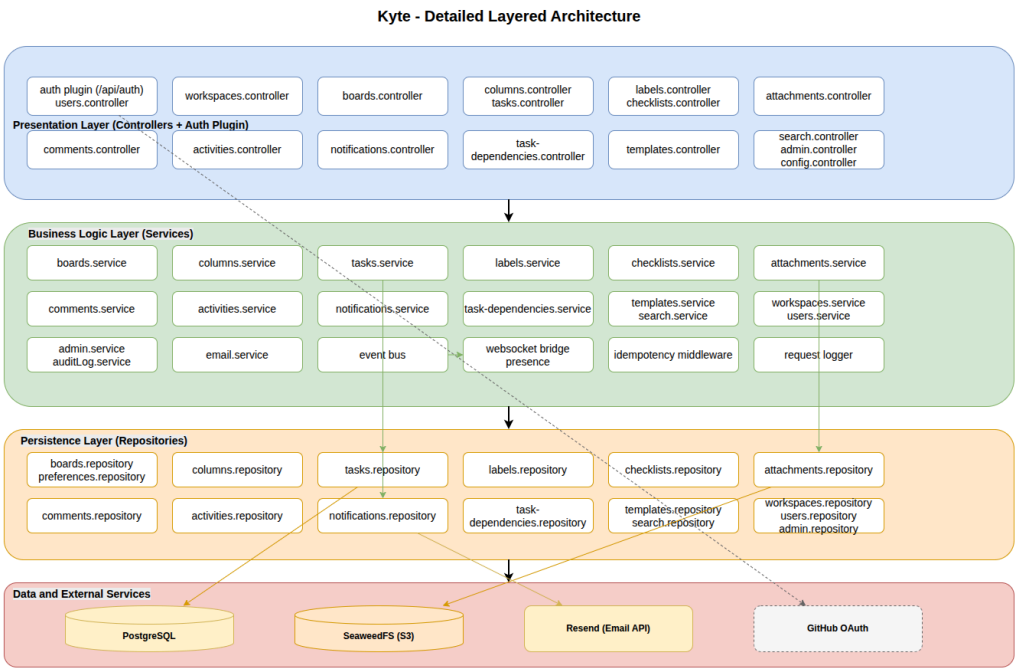
```
interface IBoardMemberRepository {
  findByBoardAndUser(boardId: string, userId: string):
         Promise<BoardMemberEntity | null>
  findByBoardId(boardId: string): Promise<BoardMemberEntity[]>
  create(memberEntity: BoardMemberEntity): Promise<BoardMemberEntity>
  delete(boardId: string, userId: string): Promise<void>
}
```

3.3 Activity Practice 3: Component Diagram Modeling

Goal: Create a visual model of the Layered Architecture using a UML Component Diagram.

3.3.1 Component Diagram Description

The UML Component Diagram for Kyte illustrates the three application layers with their components and interfaces:



Lab 2 Component Diagram

3.3.2 Diagram Key Elements

1. Layers: Three stacked rectangles representing Presentation, Business Logic, and Persistence layers, plus the Data Layer at the bottom.
2. Components: Each component is represented with the UML component notation (rectangle with two small rectangles on the left side).
3. Provided Interfaces (Lollipop —○):
    - BoardService provides IBoardService
    - ColumnService provides IColumnService
    - TaskService provides ITaskService
    - AuthService provides IAuthService
    - Each Repository provides its respective interface
4. Required Interfaces (Socket ┌):
    - BoardController requires IBoardService
    - ColumnController requires IColumnService
    - TaskController requires ITaskService
    - Services require their corresponding Repository interfaces
5. Strict Dependency Rule: All dependency arrows point strictly downward, enforcing the Layered Architecture constraint that a layer can only depend on the layer directly below it.

3.3.3 ASR Mapping to Components

ASR	Layer(s) Affected	Component(s)	Implementation Strategy
ASR-1 (Modifiability)	All Layers	All Components	Strict layer separation enables future Mobile App to use same Services
ASR-2 (Security)	Business Logic	AuthService, PermissionService	All write operations validated through PermissionService before processing
ASR-3 (Reliability)	Persistence + Business	TaskRepository, TaskService	Version field in TaskEntity; updateWithVersionCheck method; conflict handling in TaskService
ASR-4 (Scalability)	Presentation	AuthPlugin	Stateless session cookie authentication; no local server-side sessions
ASR-5 (Performance)	Persistence	All Repositories	Database indexes on board_id, column_id, user_id; efficient query patterns

4. Conclusion & Reflection

The Layered Architecture design for Kyte successfully translates the requirements from Lab 1 into a structured, maintainable system architecture. Key accomplishments include:

1. Layer Definition: Four distinct layers were formally defined with clear responsibilities, adhering to the strict downward dependency rule.
2. Component Identification: Core components for the Kanban features (Board, Column, Task management) were identified across Presentation, Business Logic, and Persistence layers.
3. Interface Design: Well-defined interfaces between layers ensure loose coupling and enable future extensibility (e.g., Mobile App development) per ASR-1.
4. ASR Alignment: Each Architecturally Significant Requirement is addressed by specific components:
    - Modifiability through layer separation

    - Security through AuthService and PermissionService
    - Reliability through optimistic locking in TaskRepository
    - Scalability through stateless session design (Better Auth)
    - Performance through indexed database queries
5. UML Component Diagram: A visual model clearly illustrates the system's logical structure with proper interface notation (provided/required).

This foundation prepares the team for Lab 3: Implementation, where these components will be realized as actual code modules following the designed architecture.

Note: The Component Diagram should be created in draw.io (Diagrams.net) using proper UML notation as described in Activity Practice 3. The ASCII representation above serves as a guide for the visual diagram creation.